Clase 22 — Taller TCP: diagramas de intercambio resueltos

Bloque: C (Transporte) · **Semana:** 8 · **Duración estimada:** 120 min **Correlativa:** Clase 21 (control de congestión en TCP) · **Sigue con:** Clase 23 (repaso de transporte + Simulacro 3) **Material de referencia:** Tanenbaum cap. 6 · RFC 9293, RFC 2018, RFC 5681, RFC 7323 · Exámenes `2011-03` P4, `2001-12` Problema 1, `2010-03` Problema 2

🎯 Al terminar esta clase vas a poder

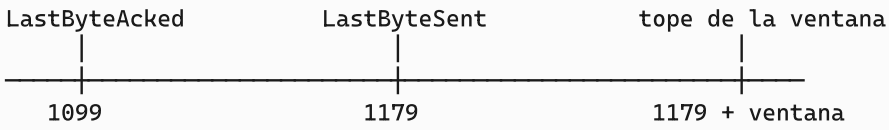
1. Dibujar el intercambio completo de N segmentos TCP **con los números a la vista**, sin pérdida.
2. Dibujar el **mismo** intercambio **con la pérdida del primer segmento**, con los ACKs duplicados y con el timeout.
3. Calcular el `cwnd` que aplica en cada caso y decir qué versión de TCP se está usando.
4. Resolver un intercambio **simultáneo en los dos sentidos** (*piggybacking*).
5. Calcular, con la cuenta a la vista, cuántos intentos necesita un **ataque TCP reset** para derribar una conexión.

> **Prerequisito del examen:** `2011-03 P4` (10 pts — ★ el problema central de esta clase), `2001-12` Problema 1 (dos conexiones y un router que pierde), `2010-03` Problema 2 (ataque TCP reset).
>
> ⚠️ **Aviso de método:** esta clase es **taller puro**: casi todo es dibujar diagramas con números. Lo único de teoría que hace falta está remindersado acá; lo demás ya lo viste en las Clases 18 a 21. Tené a mano la Clase 19 (fiabilidad, RTO) y la Clase 21 (`cwnd`).

PARTE 1 — LA MECÁNICA QUE VAMOS A USAR

Seis reglas, y con estas se resuelve cualquier diagrama de intercambio del bloque.

Regla	Qué dice	Consecuencia práctica
R1 — Seq de bytes	el `seq` es el número del **primer byte**	`seq` siguiente = `seq` actual + largo
R2 — Ack = próximo esperado	el `ack` es el **primer byte que falta**	si B recibió hasta 1119, manda `ack = 1120`
R3 — Acumulativo	un ACK confirma **todo** lo anterior	`ack = 1180` confirma los 4 segmentos
R4 — Fuera de orden ⇒ dupACK	si el byte no es el esperado, `rcv_nxt` **no se mueve**	3 veces seguidas ⇒ fast retransmit
R5 — Timeout, último recurso	solo retransmite al vencer el RTO	más lento pero siempre funciona
R6 — `ssthresh` decide el régimen	`cwnd < ssthresh` ⇒ slow start; `≥` ⇒ congestion avoidance	Clase 21 §1.3




```
bytes en vuelo = LastByteSent − LastByteAcked
nunca puede superar min(cwnd, rwnd)
```

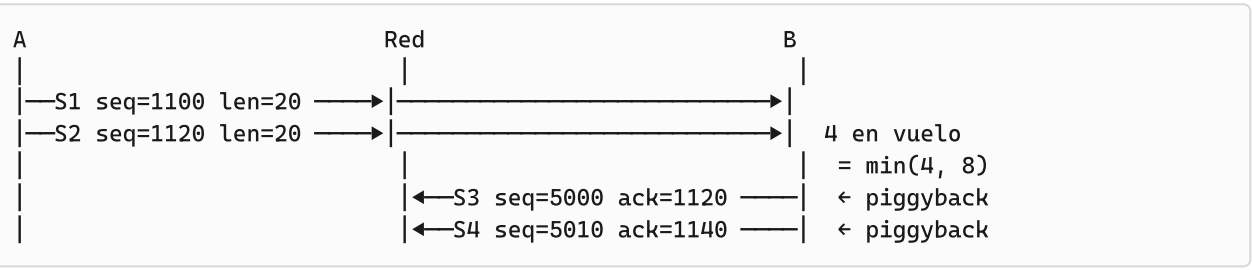
PARTE 2 — LOS SEIS EJERCICIOS

2.1 Ejercicio 1 (guía) — intercambio simple con *piggybacking*

Datos. A manda a B **dos** segmentos de 20 bytes, el primero con `seq = 1100`. `cwnd` = 4 MSS, `rwnd` = 8 MSS, **sin pérdida**.

a) Números. El `seq` avanza 20 cada vez (regla **R1**): S1 `seq = 1100` (bytes 1100…1119), S2 `seq = 1120` (bytes 1120…1139). Próximo `seq` de A = **1140**.

b) El diagrama. B tiene algo que mandar, así que **cuelga el ACK en sus propios segmentos de datos**. B manda S3 (`seq = 5000` , 10 B) y S4 (`seq = 5010` , 10 B):



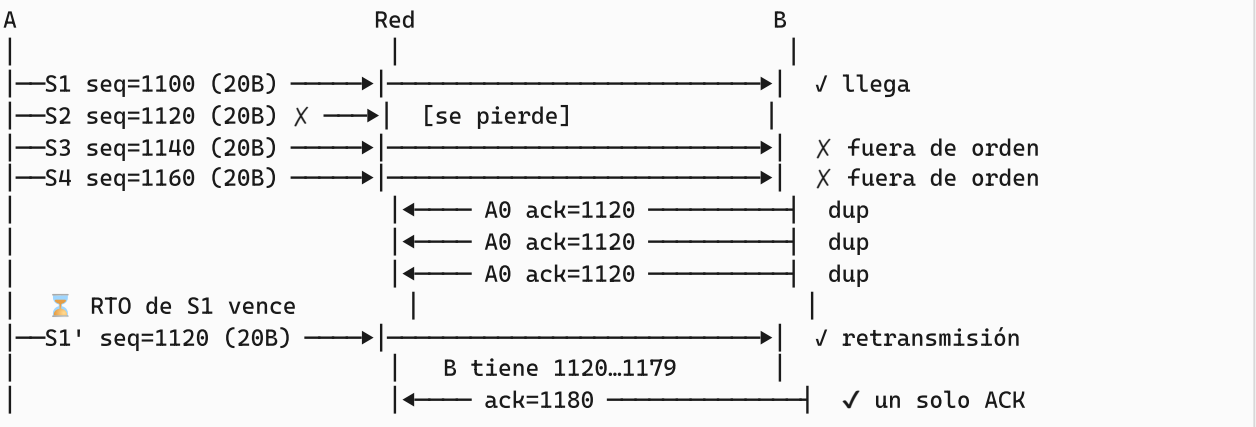
c) Los ACKs. Respuesta a S1: `ack = 1120`. Respuesta a S2: `ack = 1140` — que confirma los 40 bytes de una vez (regla **R3**).

d) ¿Cuándo queda en vuelo cero? Al recibir `ack = 1140` : `LastByteAcked = LastByteSent = 1140` ⇒ **0 bytes en vuelo**.

Interpretación. El *piggybacking* es la razón por la que un ACK vacío casi nunca se ve: si el otro lado tiene datos, los manda con el ACK pegado, ahorrando un paquete entero. ⚠️ **No confundir**: el ACK viaja **pegado al segmento de datos del sentido contrario**. El segmento de B lleva datos de B (`seq = 5000`) **y** el ACK de lo que A mandó (`ack = 1120`).

2.2 Ejercicio 2 (guía) — qué pasa con un timeout

Datos. Igual, pero **se pierde S2**.



⚠️ **Ojo:** en el caso **b) de** `2011-03 P4` el que se pierde es **S1**, así que el `ack` duplicado es `1100`, no `1120`. Acá se pierde S2 y por eso el duplicado es `1120`.

El `cwnd` (versión Reno). Con `cwnd = 4 MSS` y `ssthresh = 8 MSS`: el timeout fija `ssthresh = 4/2 = 2 MSS`, `cwnd = 1 MSS` (Tahoe y Reno coinciden) y duplica el RTO. Con el `ack = 1180`, slow start lleva `cwnd` a 2 MSS; como `2 ≥ ssthresh`, el RTT siguiente ya corre en congestion avoidance: 3 MSS, 4 MSS.

Interpretación. El diagrama oficial de la cátedra para `2011-03 P4b` dibuja el **timeout** ("vencimiento del timeout de S1, caso representado en la figura") y menciona el 3er ACK duplicado como la alternativa. **Los dos caminos** son válidos y los dos se preguntan.

⚠️ Errores típicos del taller

- **Poner `ack = 1120` cuando corresponde `ack = 1100`.** Si falta S1, B **no** puede confirmar S2: el ACK acumulativo no salta un hueco. **R4** manda.
- **Avanzar `LastByteAcked` a 1139** cuando llegó S3. No: `LastByteAcked` avanza solo con ACK.
- **Contar mal los bytes.** 4 × 20 = **80** bytes, del 1100 al **1179**, ACK final **1180**.

2.3 ⭐ Ejercicio 3 — Examen 2011-03, Pregunta 4 (10 puntos)

> **Fuente:** Examen `sirc1103` / `eirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Texto del examen (literal):**
>
> Suponga una comunicación TCP entre dos procesos A (dir IP = A; puerto TCP = 1927) y B (dir IP = B; puerto TCP = 2222). En un momento determinado A envía a B cuatro segmentos seguidos (no hay ningún dato previo sin asentar), el primero con número de secuencia igual a 1100 y con 20 bytes de datos, y los tres restantes con otros 20 bytes de datos cada uno.
>
> a) Si B no tiene datos que enviar hacia A y no hay pérdidas de segmentos: represente en un diagrama el intercambio de segmentos resultante hasta que ninguno de los dos lados tenga nada que enviar.
>
> b) Represente de nuevo el diagrama si B no tiene datos que enviar hacia A y se pierde el primer segmento enviado de A a B.
>
> *Nota: Detalle para todos los apartados anteriores el contenido de los campos relevantes de la cabecera TCP. Haga las suposiciones que considere necesarias, justificándolas.*

La nota del enunciado que lo cambia todo: "*Si suponemos que el timeout es mayor que el tiempo de llegada del 1er ACK, y las latencias son aprox. uniformes, las entidades TCP se comportan 'normalmente*'". O sea: **el timeout no vence** en el caso (a), y en el (b) lo que dispara la retransmisión es el **3er ACK duplicado**.

2.3.0 Los números, antes de los diagramas

Segmento	`seq`	Bytes que cubre	Largo
S1	**1100**	1100 … 1119	20
S2	**1120**	1120 … 1139	20
S3	**1140**	1140 … 1159	20
S4	**1160**	1160 … 1179	20

Total: `4 × 20 = 80` bytes, del **1100** al **1179**. El próximo byte que B espera es **1180**.

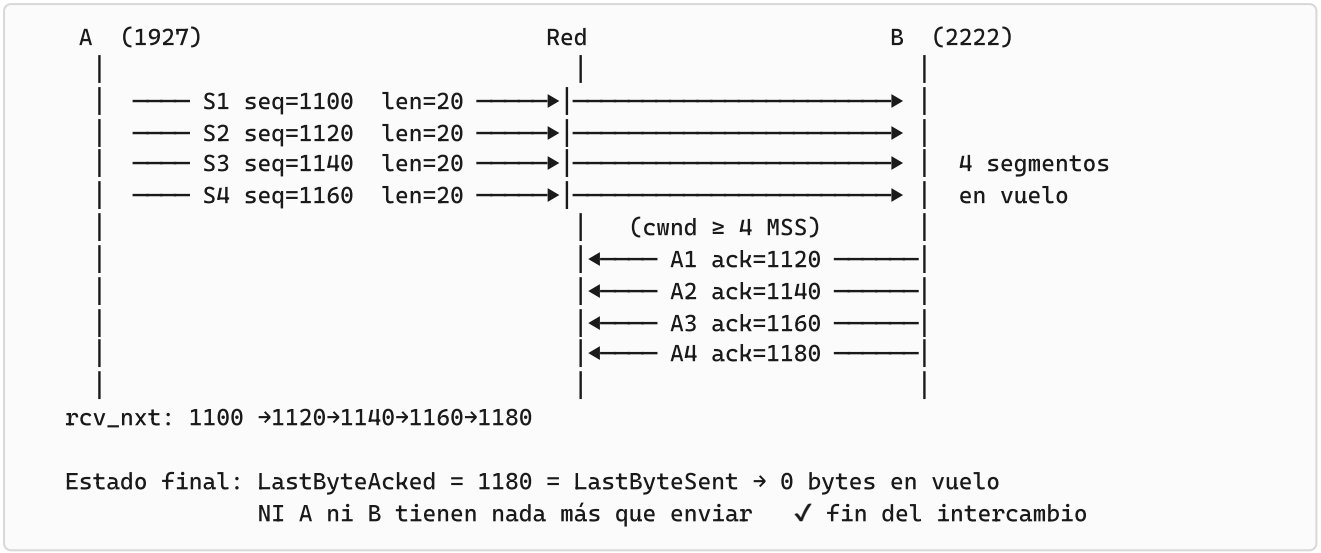
ACK	`ack`	Confirma
A1	**1120**	S1
A2	**1140**	S2
A3	**1160**	S3
A4	**1180**	S4

⚠️ **Salvedad que suma puntos:** con *delayed ACK* las confirmaciones serían **1120, 1160, 1180** (una confirma S1+S2, otra S3+S4). **La solución oficial asume un ACK por segmento.** Si dibujás con delayed ACK, **declaralo**: el resultado es equivalente porque el ACK es acumulativo.

Suposiciones que hay que declarar (el enunciado las pide explícitamente):

1. `cwnd` de A ≥ 4 MSS, así que los 4 segmentos salen **juntos** (la solución oficial no dice nada del `cwnd`, así que se asume ventana de sobra).
2. `rwnd` de B ≥ 80 bytes: B **no** es el que limita.
3. Latencias iguales ⇒ los ACKs vuelven **en orden**.
4. Timeout > tiempo del primer ACK ⇒ en (a) no hay retransmisión.
5. **Puertos**: cada segmento de A va `origen = 1927, destino = 2222`; cada ACK de B va `origen = 2222, destino = 1927`. La **4-tupla** (A, 1927, B, 2222) identifica la conexión.

2.3.1 Apartado a) — sin pérdidas



Los campos relevantes de la cabecera, como pide la nota del enunciado:

```
Segmento de A → B:
┌──────────────────────────────────────────────────────────┐
│ puerto_origen=1927  puerto_destino=2222                  │
│ numero_secuencia = <1100|1120|1140|1160>                 │
│ numero_confirmacion = <lo que B haya confirmado>         │
│ flags = ACK      longitud_ventana = rwnd de B            │
│ cantidad_datos = 20                                      │
└──────────────────────────────────────────────────────────┘

Confirmación de B → A:
┌──────────────────────────────────────────────────────────┐
│ puerto_origen=2222  puerto_destino=1927                  │
│ numero_secuencia = <nº de secuencia propio de B>         │
│ numero_confirmacion = <1120|1140|1160|1180>              │
│ flags = ACK      cantidad_datos = 0     ← no hay datos de B │
└──────────────────────────────────────────────────────────┘
```

El `cwnd` en el apartado (a). No hay pérdida, así que **el `cwnd` no baja nunca**. Cada ACK suma 1 MSS mientras esté en slow start:

```
cwnd inicial (IW) = 10 MSS   [RFC 6928, valor real de 2026]
                    o 1 MSS  [lo que dicen los apuntes de 2011 / RFC 5681]

después de A1: cwnd = IW + 1 MSS
después de A2: cwnd = IW + 2 MSS
después de A3: cwnd = IW + 3 MSS
después de A4: cwnd = IW + 4 MSS

En ningún momento hay reducción ⇒ nada de slow start, nada de congestion avoidance.
```

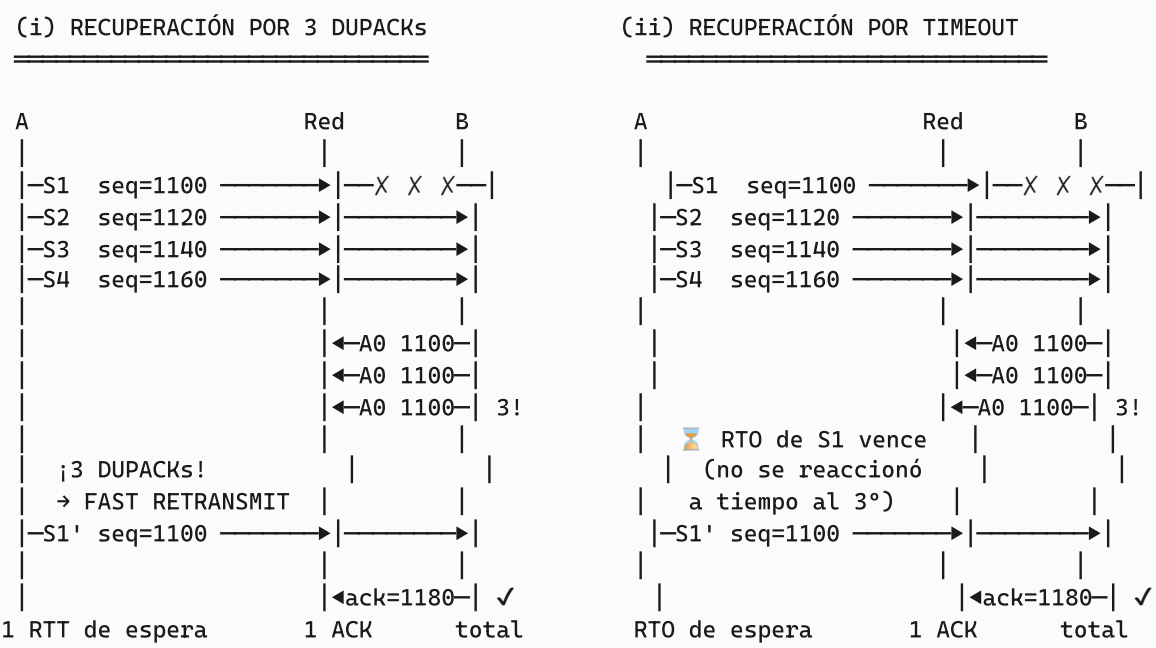
2.3.2 Apartado b) — se pierde S1

B recibe S2, S3, S4 **pero ninguno empieza en 1100**, que es lo que espera (regla **R4**). Los tres quedan fuera de orden, `rcv_nxt` **no se mueve**, y B manda el **mismo** ACK las tres veces:

```
┌──────────────────────────────────────────────────────┐
│ A0: numero_confirmacion = 1100   "sigo esperando el 1100"  │
└──────────────────────────────────────────────────────┘
```

El `ack = 1100` es **exactamente** el mismo número que habría mandado si nada hubiera llegado: no hay forma de distinguirlos, y **por diseño** no se intenta.

Los dos diagramas lado a lado:



Resultado en AMBOS: 5 segmentos (4 + 1 retransmisión) + 4 ACKs = 9 paquetes

Qué pasa cuando llega S1':

```
B tenía en su buffer de reordenamiento: S2 (1120..1139), S3 (1140..1159), S4 (1160..1179)
Llega S1' (1100..1119) → el rango 1100..1179 es CONTIGUO
→ B entrega 1100..1179 a la aplicación (ordenado, sin huecos)
→ B manda numero_confirmacion = 1180
```

Un solo ACK final, no cuatro: el ACK acumulativo confirma los 80 bytes de una vez.

2.3.3 El `cwnd` en el apartado b) — que es lo que la solución oficial pide

Evento	Interpretación	`ssthresh`	`cwnd`	Régimen
3 dupACKs	pérdida **aislada**: la red aguantó	`cwnd/2`	**`cwnd/2`** (Reno) · `1 MSS` (Tahoe)	fast recovery
Timeout	pérdida **grave**: ni el siguiente llegó	`cwnd/2`	**1 MSS** (Tahoe **y** Reno)	slow start

Con `cwnd = 4 MSS` y `ssthresh = 8 MSS` al momento de la pérdida:

```
== Variante (i): 3 dupACKs, fast recovery (Reno) ==

3 dupACKs  →  ssthresh = 4/2 = 2 MSS
              cwnd = ssthresh = 2 MSS          ← Reno: NO baja a 1
              se retransmite S1
ACK=1180   →  en fast recovery cwnd sigue en 2 MSS hasta cerrar la recuperación
RTT+1      →  2 ≥ ssthresh(2) ⇒ congestion avoidance → cwnd = 3 MSS
RTT+2      →  cwnd = 4 MSS   (vuelve al valor previo a la pérdida)
RTT+3      →  cwnd = 5 MSS


== Variante (ii): timeout ==

TIMEOUT    →  ssthresh = 4/2 = 2 MSS
               cwnd = 1 MSS                    ← Tahoe Y Reno coinciden aquí
               RTO se duplica (backoff exponencial)
ACK=1180   →  slow start → cwnd = 2 MSS
RTT+1      →  2 ≥ ssthresh(2) ⇒ congestion avoidance → cwnd = 3 MSS
RTT+2      →  cwnd = 4 MSS
```

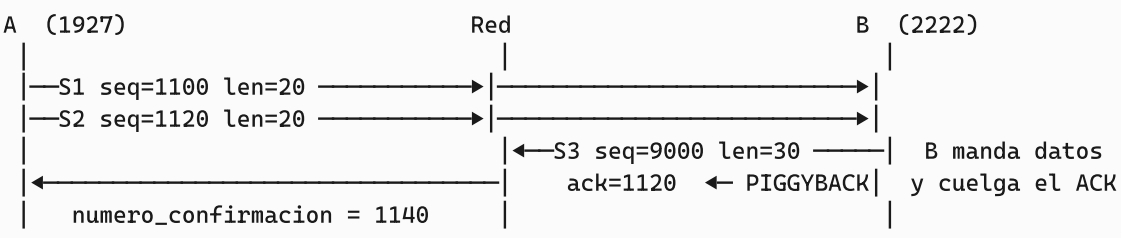
La comparación que la solución oficial quiere que veas:

```
| 3 dupACKs (Reno)        | Timeout
————————————————————————————————————————————
cwnd baja a cwnd/2 = 2    | cwnd baja a 1 MSS
Latencia: 1 RTT           | Latencia: RTO
RTO: NO se toca           | RTO: se duplica (backoff)
Recuperado: 2 MSS         | Recuperado: 1 MSS
```

> **Tope para los 10 puntos:** los cuatro `seq` correctos (1100, 1120, 1140, 1160), los cuatro `ack` del caso (a) (1120, 1140, 1160, 1180), el `ack = 1100` repetido tres veces en el (b), el **`ack = 1180` final**, la flecha de pérdida, los **puertos** (1927 / 2222) en la dirección correcta, y una frase sobre qué pasa con el `cwnd` en cada caso. **Declarar las suposiciones** vale puntos: el enunciado las pide explícitamente.

2.4 Ejercicio 4 — *piggybacking* en los dos sentidos a la vez

Datos. A y B **se mandan datos al mismo tiempo**. A manda 2 segmentos de 20 bytes desde `seq = 1100`. B manda 1 segmento de 30 bytes desde `seq = 9000`. Ninguno tiene nada más que enviar.



a) ¿Cuántos ACKs viajan? Uno solo: el ACK de S1 y S2 viaja **pegado** en el segmento de B. Si A y B hubieran hablado al mismo tiempo, habría **dos** ACKs cruzados, cada uno pegado en un segmento del otro sentido.

b) ¿Qué le pasa al `cwnd` de A? Cada ACK suma 1 MSS mientras esté en slow start.

c) ¿Y si B no hubiera tenido datos? B mandaría un **ACK vacío** (`cantidad_datos = 0`): un segmento de 40 bytes (20 IP + 20 TCP) para transportar **cero** bytes de datos, o sea una eficiencia de `4/40 = 10 %`.

Interpretación — por qué importa tanto. Un ACK vacío cuesta un paquete entero y medio RTT para transportar nada. Por eso el *delayed ACK* (un ACK cada 2 segmentos) no es solo una optimización: es la razón por la que el *overhead* de ACK es aceptable en cargas masivas.

2.5 Ejercicio 5 — Dos conexiones TCP por un router que pierde (examen 2001-12, Problema 1)

> **Fuente:** Examen `sirc0112` — 27 de diciembre de 2001. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0112.pdf **Texto del examen (literal):**
>
> Sean dos computadoras (A y B), interconectadas mediante una red TCP/IP a la que A está conectada (únicamente) mediante un router R. Desde A hay abiertas dos conexiones TCP a los ports 9001 y 9002 de B, que envían datos continuamente desde A hacia B. B únicamente contesta ACKs.
>
> En la red se producen ocasionales períodos de congestión, durante los cuales la capacidad disponible es suficiente para una de las transmisiones, pero no para ambas. Se ha determinado que es preferible "interrumpir" transitoriamente los envíos al port 9002 manteniendo inafectada la transmisión hacia el port 9001, que experimentar retrasos en ambos.
>
> La mencionada "interrupción" involucra pérdida de información (lo que no siendo que ambas transmisiones transportan voz digitalizada, estimándose aceptable que ocurran interrupciones, y en todo caso preferible a que en ambos canales haya retardos). No obstante, esta pérdida de información deberá resultar transparente a las capas de transporte de A y de B.
>
> Se pide: diseñar y codificar un proceso que ejecutará en el router R interceptando todos los paquetes correspondientes a las conexiones mencionadas, y que cumpla con el objetivo deseado. Se sugiere alterar inteligentemente los campos de secuencia correspondientes a los datos transmitidos de A hacia B y a los ACKs de B hacia A.

La respuesta oficial, textual:

> - Al comienzo de la congestión, descartar paquetes hacia B, llevando la cuenta de la cantidad de bytes descartados. Insertar ACKs hacia A (como si provinieran de B) para esos datos, o bien sumar esa cantidad al campo de secuencia de los ACKs que lleguen de B.
> - Al fin de la congestión, restar la cantidad de bytes descartados del campo de secuencia de los datos enviados hacia B (de modo que B no detecte que faltan datos), y coincidentemente sumar dicha cantidad a los ACKs de B hacia A.

La idea en una frase: alterar el espacio de numeración de la conexión 9002 para que la pérdida sea invisible. Si el `seq` que ve B no es el "real", B no puede saber que le faltó algo. Tres campos que hay que tocar:

```
┌──────────────────────────────────────────────────────────────────┐
│ 1. SN de los DATOS de A→B (puerto 9002)                          │
│    → restarle los bytes descartados acumulados (dif_SN)          │
│ 2. ACK de B→A                                                    │
│    → sumarle la misma dif_SN, para "deshacer" el desplazamiento  │
│ 3. INYECTAR ACKs falsos                                          │
│    → para que A no retransmita lo que se descartó a propósito    │
└──────────────────────────────────────────────────────────────────┘
```

El pseudocódigo oficial (`dif_SN` = bytes descartados acumulados):

```
int ProcesoServidor() {
  int next_SN, dif_SN = 0;
  pack buff, buffsend;
  while (TRUE) {
    if (status == RED_A) {
      recv_A(buff);
      if (buff.port_dest == 9001)
        send_B(buff);                          // 9001: pasa INTACTA
      else {                                   // port_dest == 9002
        if (congestion) {
          largo = largo - buff.head_length;  // largo = solo los datos
          dif_SN = dif_SN + largo;           // acumula lo descartado
          next_SN = buff.SN + largo;
          // ARMA Y MANDA UN ACK FALSO, como si fuera B
          buffsend.port_orig = buff.port_dest;  buffsend.port_dest = buff.port_orig;
          buffsend.IP_orig  = buff.IP_dest;   buffsend.IP_dest  = buff.IP_orig;
          buffsend.data     = "";
          buffsend.ACK = buff.SN + largo;           // "recibí todo"
          buffsend.SN  = buff.ACK;
          send_A(buffsend);
        } else {
          if (next_SN >= buff.SN) {
            largo = largo - buff.head_length;
            next_SN = buff.SN + largo;
          }
          buff.SN = buff.SN - dif_SN;        // <-- DESPLAZA hacia atrás
          send_B(buff);
        }
      }
    }
    if (status == RED_B) {
      recv_B(buff);
      if (buff.port_orig == 9001)
        send_A(buff);                          // 9001: pasa INTACTA
      else {                                   // port_orig == 9002
        buff.SN  = buff.SN + dif_SN;         // <-- compensa el SN
        buff.ACK = buff.ACK + dif_SN;        // <-- compensa el ACK
        send_A(buff);
      }
    }
  }
}
```

Suposiciones que el enunciado y el código oficial hacen, y hay que declarar:

1. Las conexiones ya están **establecidas** (no hay handshake que juzgue el desplazamiento).
2. Hay **una sola conexión por puerto**, y solo para los puertos **9001 y 9002**.

3. "Transparente" = **sin retransmisiones ni timeouts visibles**, **no** "sin pérdida de datos". Eso es lo que hace aceptable el esquema en voz: se pierden ~20 ms de audio, pero el control de congestión de A **no se entera**, y por eso 9001 no arrastra la caída.

2.5.1 La pregunta de fondo: ¿por qué el reparto justo no sirve?

```
RED SIN EL MECANISMO DE R (equilibrio por TCP puro):
  conexión 9001   ████████████████  ← la que NO debe sufrir
  conexión 9002   ████████████████  ← la que se debe "frenar"
  capacidad disponible: suficiente para UNA sola

El control de congestión de TCP es POR CONEXIÓN y reacciona a la pérdida de
cada una por separado: cwnd(9001) baja por las pérdidas de 9001 y
cwnd(9002) baja por las pérdidas de 9002.
⇒ "justicia" = las dos se frenan a la mitad. PEOR PARA LAS DOS.


RED CON EL MECANISMO DE R:
  conexión 9001   ████████████████████  ← intacta, sube su cwnd
  conexión 9002   ▒▒▒▒                  ← "interrumpida", transparente a TCP
9002 pierde datos pero NO retransmite y NO tira su cwnd a 1 MSS
⇒ la capacidad liberada alcanza para que 9001 vaya solo
```

Lo que hace el mecanismo, en una línea: desacoplar la señalización de congestión del transporte del estado real del medio. TCP cree que "no hubo pérdida, todo llegó" (→ `cwnd` intacto), la realidad es "se descartaron X bytes" (→ capacidad liberada), y el efecto es que **9001 aprovecha el hueco que dejó 9002**.

La sección que más se pregunta: RTT unfairness. Dos flujos por el mismo cuello de botella: si A tiene RTT = 10 ms y B tiene RTT = 200 ms, B pierde **20 veces menos seguido**, así que su `cwnd` crece más y B se lleva más ancho de banda. El reparto **no** es 50/50, aunque los dos flujos tengan igual "derecho".

¿Es injusto? Depende contra qué se compare:

Criterio	Resultado
Igualdad de tasas	**Injusto**: A y B no obtienen lo mismo
Eficiencia total	**Justo**: se aprovecha más el enlace
Percepción del usuario (un flujo lento NO frena al rápido)	**Justo**: es lo que el usuario quiere

La respuesta estándar: la **igualdad no es el objetivo correcto en redes**. Starving al flujo rápido para darle al lento la misma parte **reduce el throughput total** y perjudica a los dos. Y el ejemplo de voz es aún más claro: en voz el retardo es **peor** que la pérdida. Perder 20 ms de un fotograma es imperceptible; agregar 100 ms de cola al otro canal lo hace inservible.

⚠️ **Y un matiz técnico que suma puntos:** aunque el enunciado es de 2001, **hoy ese hack ya no es una buena idea**, por tres motivos concretos:

1. **SACK (`RFC 2018`) lo rompe**: R tendría que reescribir también los bloques SACK, que referencian los números **reales** de bytes recibidos.
2. **Timestamps + PAWS (`RFC 7323`) lo rompen**: un `SN` desplazado es detectable, porque el `TSval` no le corresponde.

3. **No es realmente transparente**: solo vale si los dos extremos usan el mismo protocolo exacto. Si uno usa SACK o un control de congestión que mire los números reales, la conexión **se rompe**.

La solución moderna es obvia una vez que la ves: en vez de **engañar** al transporte, se **marca** el tráfico y se lo prioriza en un router. **DiffServ** (Clase 40) marca el byte TOS/DSCP en la cabecera IP y el router da prioridad por **EF** (`RFC 3246`) o por **AF** (`RFC 2597`). El **marking** en el **borde** es lo que realmente permite que una conexión de voz sobreviva a un cuello de botella saturado, y a diferencia del hack **no depende del protocolo de transporte**.

2.6 Ejercicio 6 — El ataque TCP reset (examen 2010-03, Problema 2)

> **Fuente:** Examen `sirc1003` / `eirc1003` — 26 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen (extracto):**
>
> Para realizar un ataque de tipo TCP reset que termine con una conexión establecida entre los hosts A y B, un tercer host C debe fabricar un paquete que tiene como dirección IP de origen la de A y como dirección IP de destino la de B y, en su encabezado TCP tiene como puerto origen el que utiliza A en esa conexión TCP, como puerto destino el que utiliza B en ella, un número de secuencia válido (un entero de 32 bits) en la conexión activa entre A y B y el bit RST encendido. Cuando B recibe dicho paquete, inmediatamente corta la conexión con A.
>
> Para que este ataque funcione, alcanza con que el número de secuencia del paquete enviado por C sea válido dentro de la ventana en la que B está esperando paquetes. Dado que las ventanas de transmisión tienen 16KBytes y que el paquete enviado por C no incluye carga útil, se pide: … b) Si queremos atacar un cliente de un servicio conocido (p.e. un browser), pero del que desconocemos el puerto utilizado en el cliente, calcule los tiempos máximos en tirar abajo la misma conexión…

El mecanismo, paso a paso.

```
Conexión legítima A → B:   A:puerto_efímero → B:443   (una conexión TLS de un browser)

C quiere derribarla. Necesita:
1. spoofing de IP origen : poner IP_orig = la de A     ← requiere IP spoofing
2. puerto origen         : poner el efímero de A       ← hay que ADIVINARLO
3. número de secuencia   : que caiga en la ventana de 16 KB de B
4. flag RST = 1

B lo recibe, ve que cae en la ventana, se lo cree, y CORTA la conexión → DoS
```

2.6.1 a) El espacio de secuencia y cuántos paquetes

```
espacio de seq = 2³² = 4 294 967 296 números de secuencia
ventana de B   = 16 KB = 16 384 bytes = 2¹⁴

Para tapar TODO el espacio con ventanas de 2¹⁴:

paquetes = 2³² / 2¹⁴ = 2¹⁸ = 262 144
```

Cada paquete, con la cuenta a la vista:

```
Cabecera IP (sin opciones)  : 20 bytes
Cabecera TCP (sin datos)    : 20 bytes
──────────────────────────────────────
Total por paquete           : 40 bytes = 320 bits

(el enunciado dice "no incluye carga útil", así que son 40 exactos)

Total a transmitir = 2¹⁸ × 320 = 262 144 × 320 = 83 886 080 bits
                   ≈ 80 millones de bits = 10 485 760 bytes ≈ 10 MB
```

2.6.2 b) Cuántos puertos y los tiempos

```
Puertos TCP posibles = 2¹⁶ = 65 536

⇒ En el peor caso: los 65 536 puertos × los 2¹⁸ = 262 144 seq cada uno.
```

ADSL 512/128 Kbps. El atacante **inyecta** paquetes, así que usa la **subida: 128 000 bits/s**.

```
Solo los 2¹⁸ números:  t = 83 886 080 / 128 000       = 655,4 s ≈ 10,9 minutos
Los 65 536 puertos:    t = 655,4 × 65 536 = 42 949 673 s = 497 días ≈ 1,36 años
```

ⓘ **Nota:** la solución oficial escribe `625 s × 65 536 = 40 960 000 s ≈ 474 días`. La diferencia (497 vs. 474) viene de que redondean `2¹⁸ × 320` a "80 millones de bits" en vez de 83,886. **La conclusión es la misma: del orden de 1,4 años.**

Fast Ethernet (100 Mbps).

```
Solo los 2¹⁸ números:  t = 83 886 080 / 100 000 000  = 0,8389 s ≈ 0,84 s  (menos de 1 s)
Los 65 536 puertos:    t = 0,8389 × 65 536 = 54 978 s ≈ 15,3 horas
```

La solución oficial asume ~1 s por puerto: `65 536 s ≈ 18,2 horas` ("algo más de 18hs"), y con 0,1 s por puerto `6 554 s ≈ 1,8 horas`.

Escenario	Sin conocer el puerto	Con los 65 536 puertos
ADSL 512/128 (subida 128 kbps)	655 s ≈ **10,9 min**	42,9 M s ≈ **497 días (1,36 años)**
Fast Ethernet 100 Mbps	0,84 s	54 978 s ≈ **15,3 horas**

Interpretación — y la moraleja. El ataque **es factible** en una red local (15 horas) y **inviable** en Internet con los caudales de subida de 2010. Dos observaciones que valen puntos:

```
1. La solución oficial asume ~1 s POR PUERTO, no 0,84 s. ¿Por qué? Porque el
   atacante no solo genera el paquete: tiene que MEDIR si la conexión
   cayó. Ese ACK o ese timeout son los que consumen el segundo.
   En la práctica el cuello de botella es la DETECCIÓN, no la inyección.

2. El apartado (c) de 2010-03 pregunta si el puerto ALEATORIO de BSD protege.
   La respuesta oficial es:
   "Para un análisis del tipo worst-case, la selección randómica de puertos no
    nos podría ayudar a reducir el espacio de búsqueda. [...] Sin embargo,
    desde el punto de vista práctico elimina la forma de poder fácilmente
    reducir el rango de puertos a atacar."
   Es decir: en el PEOR caso no cambia nada (2¹⁶ siempre). En el caso REAL
   cambia muchísimo, porque un atacante que descubre UN puerto asignado
   puede acotar la búsqueda alrededor de él.
```

2.6.3 c) Las defensas — por qué el problema **es** un problema de números

⚠️ **Actualización a 2026:**

Sigue igual — de memoria: para matar una conexión hacen falta IP spoofing + el **puerto origen** correcto + un `seq` **dentro de la ventana** + el flag **RST**; el espacio de secuencia es de **32 bits** y la ventana típica de **16 KB** (2^{14}).

Cambió — y acá hay que ser preciso:

- **Las defensas reales no están en la ventana, están en los números.** La ventana de 16 KB sigue igual, pero:
- `ISN` **aleatorio**: el número de secuencia inicial se genera con un PRNG criptográficamente seguro, no con un contador. Sin eso el atacante **predice** el `seq` y el ataque baja de 2^{18} intentos a 1.
- **PAWS (`RFC 7323`, 2014)**: obligatorio si hay timestamps. Rechaza segmentos cuyo `TSval` sea viejo, invalidando paquetes acumulados.
- **Validación de RST (*blind reset*)**: un `RST` que llega sin datos asociados y con un `seq` que no calza exactamente con `LastByteAcked` se **ignora**. Esto es lo que más reduce el éxito: el atacante tiene que acertar **el número exacto**, no la ventana.
- **El spoofing de IP hoy es casi imposible**: la **BCP 38** (*Filtering Bogons*) obliga a los proveedores a no enviar a Internet paquetes con origen RFC1918 o de loopback, y muchos operadores hacen ***ingress filtering***: si un paquete llega con IP origen de tu LAN, se descarta. Sin spoofing, el ataque no arranca.
- **Lo que sí es un problema real hoy: los *middleboxes*.** Un `RST` inyectado funciona **perfectamente** desde el mismo lado de la conexión o desde la red local, porque no hay filtro que los separe. **Ese es el ataque que funciona en 2026**: no el hijack desde Internet, sino el `RST` desde la LAN (que es lo que hace un malware o un intermediario hostil). Y la respuesta correcta es **cifrado (TLS/DTLS) o control de acceso a nivel de red**.

PARTE 3 — EJERCICIOS EXTRA

3.1 Extra 1 — Un intercambio con **reordenamiento** (sin pérdida)

Datos. A manda 4 segmentos de 20 bytes desde `seq = 1100`. **No se pierde nada**, pero **se reordena**: S3 llega antes que S2. Sin SACK.

a) ¿Cuántos dupACKs hubo? Uno solo (el de S3). Con **3** habría fast retransmit. Con **1 o 2**, no: TCP espera.

b) ¿Y si el reordenamiento fuera mayor? Si S4 también llegara antes de S2, habría **2 dupACKs**: todavía insuficiente. Con 5 segmentos reordenados (S3, S4, S5 antes de S2) habría **3 dupACKs** ⇒ **fast retransmit sobre S2**, que en realidad **no se había perdido**. Se retransmite un segmento sano: es el costo que justifica el umbral de **3** en lugar de 1.

c) ¿Con SACK qué cambiaría?

Interpretación. El **reordenamiento es el enemigo natural del fast retransmit**: es la razón del umbral 3 y la razón por la que SACK importa tanto.

3.2 Extra 2 — Tiempo total de una transferencia con ventana y RTT dados

Datos. Archivo de **8 MB** (`8 · 10⁶` bytes). `rwnd = 256 KB = 262 144 B`, `cwnd = 1 MB` (manda el `rwnd`). RTT = **40 ms**. Enlace = **100 Mbps**. Sin pérdidas.

a) Ventana efectiva: `min(1 048 576, 262 144) = 262 144 bytes`.

b) Throughput:

```
por ventana:  262 144 / 0,040 = 6 553 600 B/s = 52,4 Mbit/s
por enlace:   100 · 10⁶ / 8  = 12 500 000 B/s = 100 Mbit/s
throughput = min(...) = 6 553 600 B/s   ← manda la ventana
```

c) Tiempo: `t = 8 · 10⁶ / 6 553 600 = 1,2207 s ≈ 1,22 s`.

d) Si `rwnd = cwnd = 4 MB`:

```
por ventana:  4 194 304 / 0,040 = 104 857 600 B/s = 838,9 Mbit/s
por enlace:   100 Mbit/s  ← ahora manda el ENLACE
t = 8 · 10⁶ / 12 500 000 = 0,64 s
```

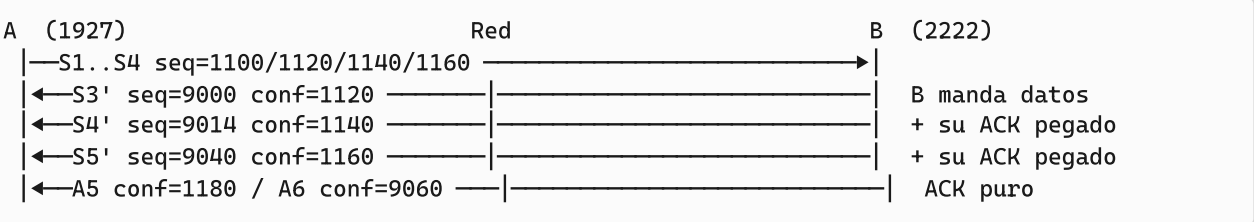
e) ¿Cuál es la ventana que hace falta para gastar los 100 Mbps? `BDP = 100 · 10⁶ bits/s × 0,040 s = 4 · 10⁶ bits = 500 000 bytes`.

Configuración	Ventana efectiva	Throughput	Tiempo para 8 MB
`rwnd = 256 KB`	262 144 B	52,4 Mbit/s	**1,22 s**
`rwnd = cwnd = 4 MB`	4 194 304 B	100 Mbit/s	**0,64 s**
`rwnd = cwnd = BDP = 500 KB`	500 000 B	100 Mbit/s	**0,64 s**

Interpretación — y el detalle fino que da puntos. La fila 2 muestra que **agrandar la ventana más allá del BDP no sirve**: con 4 MB te da lo mismo que con 500 KB, porque el **enlace** es el techo. Pero para *llegar* a ese techo la ventana tiene que estar **por encima** del BDP durante un rato, porque al principio `cwnd` es chico. Y en la práctica la transferencia tarda más que 0,64 s: los primeros RTTs van con ventana chica mientras `cwnd` crece por *slow start*.

3.3 Extra 3 — full duplex: B manda datos mientras A sigue mandando

Datos. A manda 4 segmentos de 20 bytes desde `seq = 1100`. B, en vez de solo asentar, **manda su respuesta** en 3 segmentos desde `seq = 9000`. Full duplex: los ACKs viajan **cruzados** (el `conf = 1120` de B confirma datos de A; el `conf = 9000` de A confirma datos de B), y cuando B se queda sin datos que mandar aparece un **ACK "puro"** (`cantidad_datos = 0`).



Lo que hay que ver: (1) los `seq` son **independientes por dirección** (A numera 1100..1179, B numera 9000..9060): son dos espacios distintos, así que ambos pueden tener el mismo `seq` sin conflicto; **(2)** cada sentido tiene su **propia ventana**, y cada segmento lleva el **`rwnd` del otro**; **(3)** aparece un ACK "puro" cuando se acaba un lado; **(4)** ningún lado espera. Totales: `A→B = 4×20 = 80` (confirmado con `conf = 1180`) y `B→A = 3×20 = 60` (con `conf = 9060`).

Interpretación. Esta es la situación real de **cualquier** transferencia HTTP: el cliente pide, el servidor responde, y los dos mandan datos al mismo tiempo con los ACKs pegados. Si en un diagrama de examen dibujás los ACKs como flechas separadas, casi seguro estás mal.

📌 Resumen

1. Las **6 reglas** resuelven cualquier diagrama del bloque: `seq` de bytes, `ack` = próximo esperado, ACK acumulativo, fuera de orden ⇒ dupACK, timeout es el último recurso, `ssthresh` decide el régimen.
2. `seq` siguiente = `seq` actual + largo. Nada más hay que saber para calcular números.
3. **`2011-03 P4a`** : 4 segmentos (1100, 1120, 1140, 1160), 4 ACKs (1120, 1140, 1160, 1180), fin con 0 bytes en vuelo.
4. **`2011-03 P4b`** : se pierde el primero ⇒ B manda **`ack = 1100`** **tres veces**, y al recibir el retransmitido contiguo manda **`ack = 1180`** .

5. **3 dupACKs ⇒ fast retransmit** (Reno: `cwnd/2`); **timeout ⇒ slow start** (`cwnd = 1 MSS`) y el RTO se duplica.
6. **Piggybacking**: el ACK viaja pegado a un segmento de datos **del sentido contrario**; ahorra un paquete entero.
7. **Dos conexiones + router que pierde**: se **desplaza el espacio de numeración** (`dif_SN`) para que la pérdida sea transparente. La asimetría es **a propósito**: optimiza eficiencia y percepción, no igualdad.
8. **Ataque TCP reset**: $2^{32} / 2^{14} = 2^{18} = 262\,144$ paquetes por puerto; con $2^{16} = 65\,536$ puertos, ≈ **497 días** por ADSL o **15 horas** por Fast Ethernet.
9. La defensa no es la ventana: es el `ISN` **aleatorio**, **PAWS**, la **validación de RST** y el ***ingress filtering*** contra el spoofing.
10. **Reordenamiento** y **piggybacking** son los dos casos que más rompen los diagramas "de libro" — y los dos se preguntan.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. A manda un segmento con `seq = 5000` y 30 bytes. ¿Qué `seq` lleva el siguiente?

<details>
<summary>Respuesta</summary>
</details>

2. B recibió S1 (seq 1100, 20 B) y S3 (seq 1140, 20 B), pero no S2. ¿Qué `ack` manda y qué hace con S3?

<details>
<summary>Respuesta</summary>
</details>

3. En `2011-03 P4b`, ¿cuántos segmentos y cuántos ACKs viajan en total?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Qué diferencia hay, en el `cwnd`, entre recuperar por 3 dupACKs y por timeout?

<details>
<summary>Respuesta</summary>
</details>

5. En el problema de `2001-12`, ¿por qué hay que alterar **los dos** campos?

<details>
<summary>Respuesta</summary>
</details>

6. Ataque TCP reset con ventana de 16 KB. ¿Cuántos paquetes hay que mandar si conocemos IP y ambos puertos? ¿Y si además no conocemos el puerto?

<details>
<summary>Respuesta</summary>
</details>

7. ¿Eligió BSD puertos aleatorios: eso protege contra el ataque?

<details>
<summary>Respuesta</summary>
</details>

8. ¿Qué pasa con el `cwnd` si el problema del examen 2011-03 P4 lo modelamos con SACK activado?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1103` — P4 (10 pts): ★ 4 segmentos de 20 bytes desde `seq = 1100`, con y sin pérdida del primero · Problema 1 (30 pts): ECN (Clase 21)
- **Examen** `sirc0112` — Problema 1: dos conexiones TCP por un router que pierde · Problema 2: el flag PUSH
- **Examen** `sirc1003` — Problema 2 (30 pts): ataque TCP reset, rango de puertos y tiempos
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.5–6.7 (pág. ~590–610 del PDF)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 5681` — TCP Congestion Control (2009)
- `RFC 2018` — SACK (1996) · `RFC 7323` — Timestamps, Window Scale y PAWS (2014)
- `RFC 6928` — Initial Window = 10 MSS (2013)
- **Clase 19** — fiabilidad: RTO, Jacobson/Karels, BDP, PSH · **Clase 21** — control de congestión
- **Clase siguiente:** 23 — Repaso de transporte + Simulacro 3